TreeLock Design Document

S. C. Dunn

June 11, 2025

Project: TreeLock

Subtitle: *Merkle Tree-Based Inheritance in Decentralized Module Systems*

1 Data Structure Design

Tree Representation

- **Decision:** Use a binary tree with `leaf` and `node` constructors.
- **Rationale:** Maps cleanly to Merkle semantics; enables recursive, verifiable hashing.

```
inductive Tree ( : Type)
  | leaf :  → Tree
  | node : Tree  → Tree  → Tree
```

2 Hashing Strategy

Polymorphic Tree Hashing

- **Decision:** Define `Hashable (Tree )` given `Hashable` .
- **Encoding:**
  - `leaf x` → `hash [0x00, x]`
  - `node l r` → `hash [0x01, hash l, hash r]`
- **Rationale:** Ensures structure is encoded cryptographically.

3 List to Tree Conversion

Balanced Conversion

- **Decision:** Split list in half recursively rather than flattening.
- **Rationale:** Guarantees balanced Merkle structure and efficient updates.

```
def listToTree : List Hash → Tree Hash
  | []      => Tree.leaf defaultHash
  | [x]     => Tree.leaf x
  | xs      =>
      let mid := xs.length / 2
      Tree.node (listToTree xs[..mid]) (listToTree xs[mid..])
```

4 Merkle Hash Function

Tree-Based Merkle Hash

- **Decision:** Canonical definition via `Tree Hash`.
- **Rationale:** Simplifies termination and proof strategy.

```
def merkleHash (xs : List Hash) : Hash :=
  hash (listToTree xs)
```

5 Formal Verification in Lean

Theorems Proven

- `merkleHash [] = defaultHash`
- `merkleHash [x] = hash [0x00, x]`
- `merkleHash [x, y] = hash [0x01, hash [0x00, x], hash [0x00, y]]`

Pending

- Proof of consistency: `hash (listToTree xs) = merkleHash xs`
- General correctness of listToTree structure

6 Module System Integration (Tangl)

Override as Tree Update

- **Decision:** Symbol overrides modeled as tree mutations.
- **Rationale:** Maintains integrity and traceability.

Recursive Lineage Tracking

- **Decision:** TreeHash used as provenance signature.
- **Rationale:** Enables reproducibility and diffing.

7 Hash Encoding Format

- **Leaf tag:** `0x00`
- **Node tag:** `0x01`
- **Rationale:** Avoids collisions and domain confusion.

8 Prototyping and Implementation

- **Language:** Lean 4 (for executable proofs)
- **Prototype Frontend:** Tangl (mentioned only in prototype scope)

9 Documentation and Paper Integration

- **Appendix:** Appendix B describes formal connections to module systems.
- **Main Text:** Figures visualize tree hashing. Notation consistent across Lean + prose.

10 Open Questions

- Generalization to non-binary or variadic branching trees
- Efficient diffing and reconciliation between tree versions
- Reusable tree patch or override languages